Analiza Performanței Algoritmului de Clustering K-Means

Scop: Evaluarea scalabilității algoritmului K-Means pe seturi de date masive (10.000 - 100.000 instanțe).

- **Obiectiv:** Compararea timpilor de execuție între varianta secvențială (CPU) și tentativele de accelerare hardware (GPU/Parallel).

2. Metodologia de Testare

- **Generarea Datelor:** S-a utilizat funcția `make_blobs` din biblioteca Scikit-Learn pentru a crea seturi de date sintetice cu 2 caracteristici (X, Y) și 4 centri de greutate.
- **Reproductibilitate:** Fixarea parametrului `random_state=42` pentru a asigura aceleași condiții de testare pe toate arhitecturile.
- **Hardware utilizat:**
  - CPU:Ryzen 5 3600 (6core)
  - GPU: NVIDIA GeForce RTX 2060.
  - Software: Python 3.9, Miniconda, VS Code.

3. Etapa 1: Varianta Secvențială (Baseline)

- **Rezultate inițiale (10.000 instanțe):** 1.8481 secunde.
- **Analiza performanței:** S-a observat un fenomen de "Cold Start". La prima rulare, timpul este ridicat din cauza încărcării bibliotecilor în memorie și a stării de consum redus a procesorului.
- **Rezultate stabilizate (Warm-up):** După rulări consecutive, timpul pentru 10.000 instanțe a scăzut la **1.39 secunde**. La fel am observat și pentru 50.000 de instante

Primele rulari

Instane	Timp
10k	1.8481
50k	1.5440
100k	1.4491

Tesatarea pentru 50k instante

Instane	Timp

50k	1.5514
50k	1.4139
50k	1.3839

4. Analiza Scalabilității

- **50.000 instanțe:** 1.55s (prima rulare) -> 1.38s (stabilizat).
- **100.000 instanțe:** 1.46s.

Concluzie intermediară: Algoritmul secvențial prezintă un plafon de performanță. Creșterea volumului de date de 10 ori (de la 10k la 100k) nu a dus la o creștere liniară a timpului, ceea ce sugerează că procesorul utilizează deja optimizări de instrucțiuni vectoriale (AVX), dar este limitat de frecvența unui singur nucleu sau de managementul memoriei.

5. Implementarea Variantei Secvențiale (CPU)

Pentru stabilirea punctului de referință (baseline), s-a utilizat biblioteca standard `scikit-learn`. Implementarea pune accent pe izolarea timpului de calcul matematic de cel de generare sau vizualizare a datelor.

5.1. Generarea Setului de Date

```
from sklearn.datasets import make_blobs

# Generarea a 100.000 de puncte în spațiu 2D, distribuite în 4 clustere
X, y = make_blobs(n_samples=100000, centers=4, n_features=2, random_state=42)
```

5.2. Algoritmul K-Means și Măsurarea Performanței

S-a utilizat clasa `KMeans` cu parametrul `n_init='auto'` pentru o inițializare eficientă a centroizilor. Măsurarea timpului s-a realizat folosind biblioteca `time` pentru a capta latența exactă a metodei `.fit()`.

```python
import time
from sklearn.cluster import KMeans

# Inițializarea modelului
model_cpu = KMeans(n_clusters=4, n_init='auto', random_state=42)

# Măsurarea timpului de execuție efectiv
start_time = time.time()
model_cpu.fit(X)
end_time = time.time()

duration = end_time - start_time
print(f"Timp execuție: {duration:.4f} secunde")
```

4.3. Vizualizarea Rezultatelor

Pentru validarea corectitudinii clustering-ului, s-a generat un scatter plot unde culorile reprezintă grupurile identificate de algoritm.

```python
import matplotlib.pyplot as plt

plt.scatter(X[:, 0], X[:, 1], c=model_cpu.labels_, s=1, cmap='viridis')
plt.title(f"Rezultat Clustering CPU (Timp: {duration:.4f}s)")
plt.show()
```